

模型只是 越做越大嗎？

從 GPT-1 走到今天的推理模型：規模、資料，以及「把任務放在哪裡」這件事怎麼改變。

—— 張睿明 / AIMing Data



CHANG JUI-MING



本堂路線

01

從 GPT-1 到 GPT-3

架構沒換，變的是規模、資料，以及任務放在哪裡。

中段會看兩份 *notebook* 的實際輸出。

02

模型吃什麼長大

預訓練資料怎麼被清出來：去重、去個資、去重複話。

03

讓 AI 把過程寫出來

Chain-of-Thought 與 Reasoning Model 的差別。差別。



AI News this week | 這禮拜的三則

開場暖身：先看看這禮拜 AI 圈冒出來的東西。

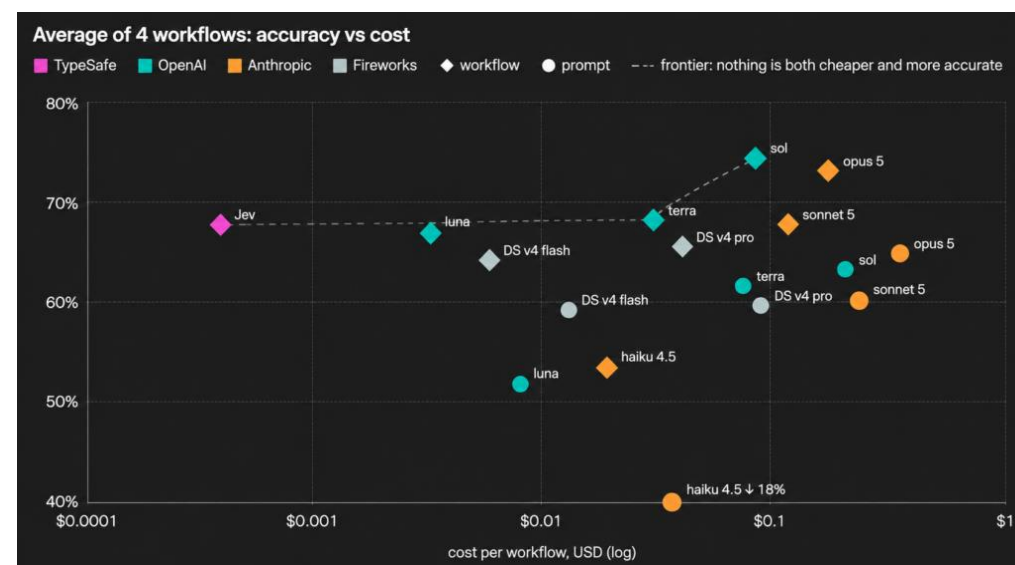
01 · GROK BOT GALAXY

Grok Bot Galaxy

Grok Bot is your team of extraordinarily capable AI agents. See it in action at this three day event featuring live demos and hands-on build sessions with the team behind the product.

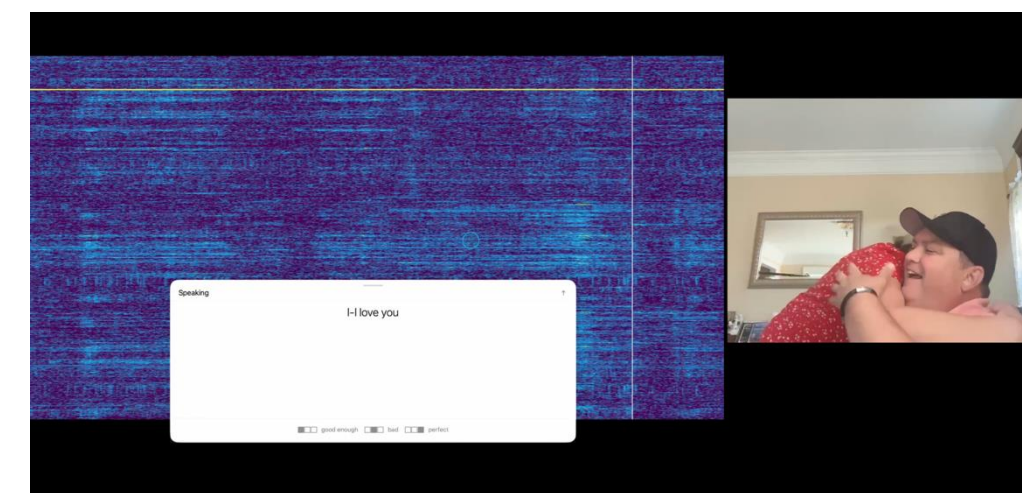
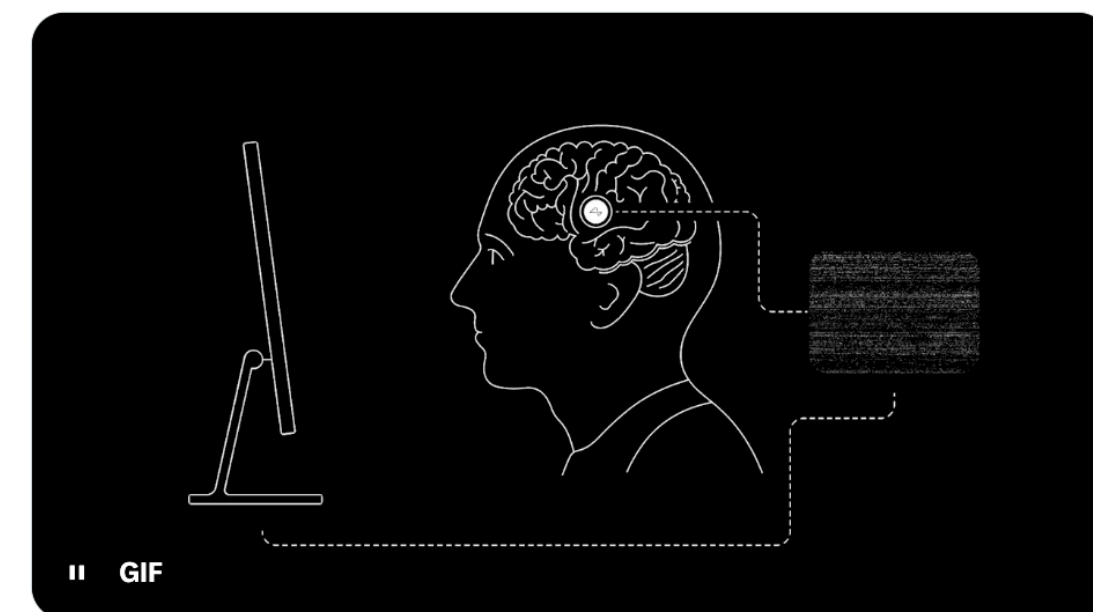
xAI 的 agent 團隊，三天的 live demo 與實作工作坊。

02 · JEV



四種 workflow 的準確率 vs 成本：前緣上沒有「又便宜又準」的選項。

Neuralink 植入物擁有超過 1,000 個電極，分佈在超過 100 條線路上，每條線路比人類頭髮還細，並由我們的 surgical robot 插入運動皮質。這些電極記錄神經元在參與者意圖移動時發射的活動。



<https://x.com/neuralink/status/2100993363403063404?s=20>

Neuralink 釋出的最新展示影片。

[X.COM/NEURALINK](https://x.com/neuralink)

<https://x.com/v0xium/status/2101526107128529120>



上週回顧：輸入 → 運算 → 輸出

01	02	03	04	05
切成 Token	轉成向量	Attention	疊很多層	預測下一個
文字先切成 Token，模型看 看不到字母	每個 Token 變成數字，加上 位置	互相對照上下文，決定誰 重要	同樣運算疊幾十層，關係 越抽象	輸出機率分布，挑一個再往 下接

經典案例：Strawberry 有幾個 r？模型看到的是 Token，不是字母——所以它會數錯。
這是 2024 年的案例，現在的推理模型大多已經數對了。



01 從 GPT-1 到 GPT-3

不是突然換了一顆腦，而是同一種腦開始讀更多資料、長得更大。



GPT 是哪三個字

G

Generative

可以生成內容

P

Pre-trained

先經過大規模預訓練

T

Transformer

採用上一場談的架構

2018 年的第一代論文標題其實沒有寫「GPT-1」——當時也沒人知道後面會數到哪裡。



GPT-1：先把語言學起來，再準備不同考試

論文當時用微調，後面會介紹後訓練

STAGE 1

預訓練

用大量沒有標籤的文字，練習預測下一個 Token。像先讓一個人人廣泛閱讀，建立語感。



STAGE 2

微調

用較小、有標籤的資料適應特定任務。像臨時抱佛腳做短期衝刺，參數會被更新。

1.17 億個參數(訓練可調整)

7,000+ 本未出版書籍 (BooksCorpus)

$$Y = Ax + B$$

A、B 是模型透過訓練學到的「參數」

訓練完成後，這些參數會被保存下來，成為模型的一部分。



標籤資料像字卡的正反面

正面是題目，翻到背面才印著標準答案。微調需要的就是這種一問一答的成對資料。

因為標註有助於模型準確度，這幾年也多了不少「資料標註員」的遠端工作。

INPUT

一張貓咪的照片



LABEL

「貓」

Go to

<https://huggingface.co/datasets/data-is-better-together/fineweb-c/viewer/default/train?p=1&row=136>



GPT-2：把模型放大，把資料增加

15 億 (1.5B)

最大版本的參數量，是 GPT-1 的十倍以上

800 萬

份網路文件，約 40 GB 文字 (WebText)

3 karma

資料來自 Reddit 上被按讚過的外部連結

比書本資料多了一層篩選：曾經有人覺得這個連結值得按讚。模型吃什麼長大，多少會反映在它後來會做什麼。



Zero-shot 不是無中生有

提示喚起了模型在訓練資料中曾經學過的條件結構。

問題後面接著答案

英文句子後面接著翻譯

長篇文章後面接著摘要

一段文字後面接著評論或分類

所以模型能力不只由參數量決定，也受訓練資料裡有哪些語言、知識與任務結構影響。

Demo: 用川普的語氣改寫新聞



Sakana Chat

<https://chat.sakana.ai> · [翻譯這個網頁](#)

Sakana Chat

毎日の調べものから、アイデアの壁打ちまで.



GPT-3：不改參數，只在上下文裡示範

1,750

億個參數

Zero-shot

只給任務指令，不給任何示範

One-shot

先提供一組輸入與輸出的示範

Few-shot

提供少數幾組示範，讓模型抓規律



Few-shot 的實際寫法

[1] 全面加徵 25% 進口鋼鐵關稅！ → 大盤下跌

[2] 科技大廠宣布在美擴建新廠，釋出數千就業機會！ → 大盤上漲

[目標] 今天與主要貿易夥伴順利達成歷史性降稅協議！ → ?

模型看懂前兩組的對照關係後，就很有機會推出最後一題應該接什麼。這個過程稱為 **In-context Learning**：參數沒有更新，改變的只是這一次輸入的 Context。

用新聞做Demo



上課，還是便利貼

FINE-TUNING

真的去上課

模型內部參數被修改，比較像長期學習。

IN-CONTEXT LEARNING

桌上貼便利貼

參數沒有改變，考前照著幾題範例作答。

對話結束、Context 消失後，那張便利貼也跟著消失了。



真正改變的是任務放在哪裡

MODEL	SCALE	DATA	任 務 放 在 哪 裡
GPT-1	1.17 億	BooksCorpus	微調資料裡
GPT-2	15 億	WebText	從大量文字中認出任務形式
GPT-3	1,750 億	更大更多元的語料	指令與範例直接放進 Context



Scaling Law 說了什麼

2020 年 Kaplan 等人觀察到：語言模型的測試損失，會隨著參數量、資料量與計算量以 **幂律** 穩定下降，而且在好幾個量級之內可以事先預測。

這件事的產業意義很大：先用小模型量出曲線，就能估算「花多少錢、能換到多少品質」。

N · 參數量

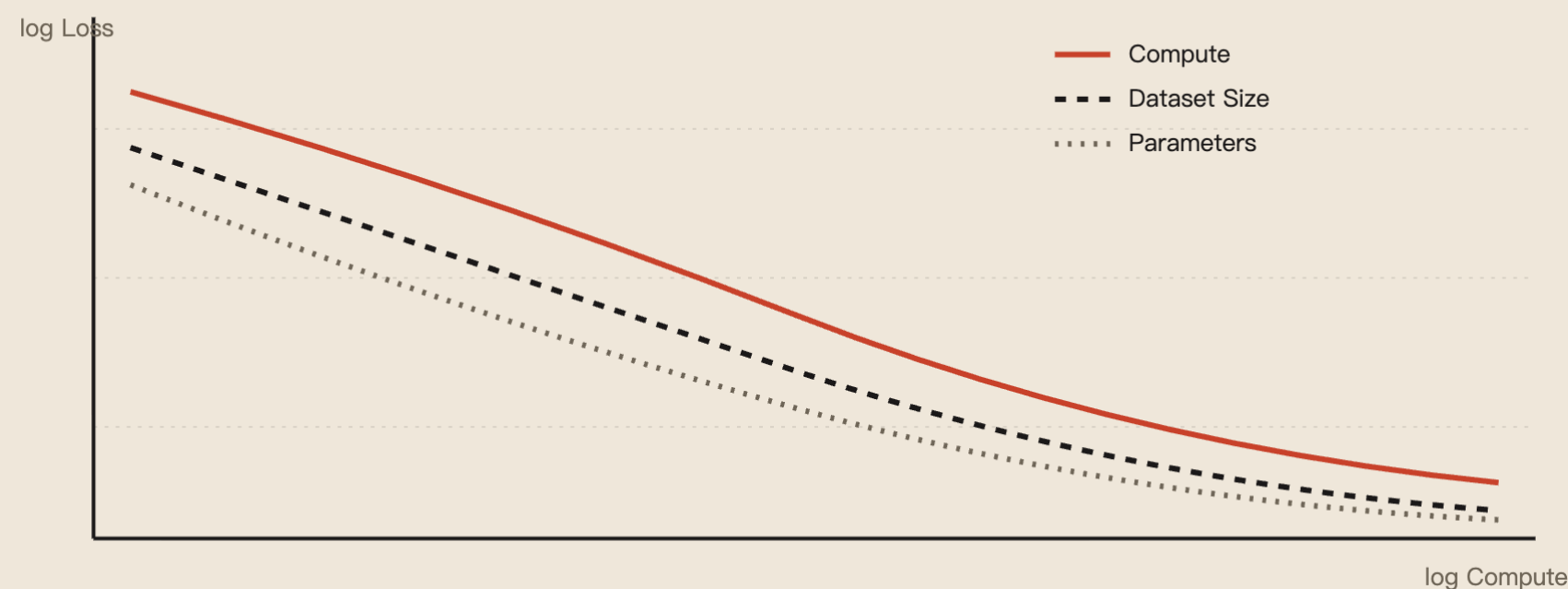
D · 訓練資料量

C · 計算量

LOSS

三者放大，損失就下降——但下降的是「預測下一個 Token 的誤差」。

N 由模型結構決定：層數 × 寬度（ d_{model} ）。
 $C \approx 6 \times N \times D$ ，算力就是參數量 × 看過的 Token 數。



後來的修正：多數大模型其實吃太少

之前的做法

參數拚命放大

資料量沒有跟上，模型處於「訓練不足」的狀態。

Chinchilla 的結論

參數與資料要等比例

經驗法則：每 1 個參數，大約配 20 個訓練 Token。

70B > 280B 同樣的計算預算下，較小但吃得夠飽的模型，表現可以勝過大它四倍的模型。



那 Scaling Law 是對的嗎？

它是一條經驗規律，不是物理定律——是趨勢，不是保證書。

01 它預測的是損失，不是我們在意的能力。損失下降不等於答案正確。

02 高品質文字資料是有限的，資料這一軸不能無限放大。

03 冪律代表報酬遞減：想再進步一點，成本要再翻好幾倍。

01 資料品質與架構會改變曲線的常數，同樣的規模不一定同樣的結果。

所以這幾年的重心，從「把預訓練做得更大」，逐漸移到後訓練與「回答時多花一點計算」——那正是下半場要談的推理模型。



現在看模型大小， 要看兩個數字

DENSE · 稠密

每個 Token 都跑完整個模型

總參數就是實際算力。Llama 3.1 405B 是目前最大的全稠密開源模型，對硬體要求極高。

MoE · 混合專家

只叫出需要的那幾位專家

總參數是知識庫大小，激活參數才決定這次要花多少算力——第一場那個 MoE 玩笑，其實是真的。

開源／開放權重	總參數	激活參數	架構
DeepSeek R1 系列	1.6 兆	約 49B	MoE
Kimi K2.6	1.1 兆	部分激活	MoE · 長文本
Qwen3 235B	235B	約 22B	MoE
Llama 3.1 405B	405B	全部 405B	Dense

閉源旗艦不公布參數，業界推估 GPT-4 約 1.8 兆的 MoE、更新世代可能到數兆級——這些是推估值，不是官方數字。想在自己電腦上跑落在 24B–31B；我們練習用的 Qwen2.5-0.5B 更小。



和 AI 溝通時，真正重要的只是把問題寫得很長嗎？

任務

要做什麼

範例

長什麼樣子

限制

不能怎麼做

預期輸出

交回什麼格式

比長度更重要的，是這四件事有沒有寫清楚。模型不是因為看到一篇落落長的作文就突然感動，而是從 Context 中尋找可以延續的模式。



02

模型吃什麼長大

網路上抓來的文字不能直接餵。來看一次最小規模的清理流程。



預訓練資料的清理流程

01	02	03	04	05
去標籤	篩語言	去重複	遮個資	去贅句
把 HTML 標籤與多餘空白清掉	只留下目標語言的文件	MinHash 找出高度相似的文件	Email、卡號、電話換成標記	壓掉重複刷出來的同一句話

真實規模大得多，但步驟的種類差不多就是這五類。清理的每一個決定，都在決定模型會學到什麼。



去掉標籤，再判斷語言

```
stripped = BeautifulSoup(text, "html.parser").get_text(" ")
stripped = re.sub(r"\s+", " ", stripped).strip()
han = len(re.findall(r"[\u4e00-\u9fff]", stripped))
kana = len(re.findall(r"[\u3040-\u30ff]", stripped))
if han > kana: keep(stripped)
```

OUTPUT

<p>今天 天氣 很好 ！ </p>



今天 天氣 很好 ！

用漢字與假名的數量差來猜語言，快但粗糙——中日夾雜的文件就會被判錯。



去掉重複， 遮掉個資

MINHASH LSH

```
lsh = MinHashLSH(threshold=0.7, num_perm=128)
if not lsh.query(m):
    lsh.insert(i, m); unique.append(text)
```

用字元組合估相似度，相似度超過門檻就視為重複。重複資料會讓模型過度記住同一段話。

開個html來測試看看

PII MASKING

```
re.sub(r"[\w.-]+@[\w.-]+", "[EMAIL]", t)
re.sub(r"\b\d{13,19}\b", "[CREDIT_CARD]", t)
re.sub(r"09\d{2}[-\s]?\d{3}[-\s]?\d{3}", "[PHONE]", t)
```

Email、卡號、手機換成標記。這只是最基本的一層——遮不掉的個資，模型就有機會背下來。



03

讓 AI 把過程寫出來

Chain-of-Thought 與 Reasoning Model：一個是個是用法，一個是模型。



一件商品原價 1,000 元，先打八折，再加上折後價格 5% 的稅金，最後是多少？

直接回答

840 元

答案對了，但我們不知道它怎麼算的。

展開步驟

$$1,000 \times 0.8 = 800$$

$$800 \times 0.05 = 40$$

$$800 + 40 = 840$$

多了一段中間過程，我們才知道它是怎麼走到答案的。



Chain-of-Thought 不是新的模型架構



沒有替 Transformer 加上新零件，也沒有插入一顆邏輯晶片。模型仍然在預測下一個 Token，只是多取得了一些可以用來計算、檢查與延續模式的 Token。

原本要心算，現在至少發了一張計算紙。至於會不會算錯，仍然是另一回事。



最初的做法：給示範

問題：小明有 3 顆蘋果，又買了 2 顆，
總共有幾顆？

推理：原本有 3 顆，又增加 2 顆。
 $3 + 2 = 5$ 。

答案：5 顆。

再給新題目，讓模型模仿同樣的回答結構。

後來發現：一句話也行

Let's think step by step.

不提供完整示範，只在問題後加一句，也可能提升大型模型在部分多步驟任務上的表現。這句話當時幾乎成了 Prompt 世界裡的都市傳說。



它不是萬用咒語

值得展開

需要草稿紙的題目

計算、規劃、比較多個條件、多步驟判斷。

判斷標準很簡單：這一題，你自己會不會想拿紙筆算一下？

不必展開

只需要一句答案的題目

知識查詢、翻譯、格式轉換。硬要長篇推理只會更慢、更長。



兩種問法， 只差一句話

```
def ask_direct(q):  
    return generate(f"{q}\n請直接只回答最終答案， 不要說明過程。", max_new_tokens=30)  
  
def ask_cot(q):  
    return generate(f"{q}\n  
請一步一步思考， 寫出你的推理過程， 最後一行用「答案：」加上最終答案。", max_new_tokens=250)
```

直接回答

又快又短，但在多步驟題目上容易直接算錯。

Chain-of-Thought

字數與時間都是好幾倍，正確率通常較高——但不保證。

提醒：notebook 只跑 3 題，是一次示範，不是嚴謹的統計實驗；重跑一次的數字也不會完全一樣。



CoT 是一種用法， Reasoning Model 是一種模型

面 向	CHAIN-OF-THOUGHT	REASONING MODEL
核心概念	用 Prompt 引導模型展開步驟	以特別後訓練與更多推論計算處理複雜任務
需要特定模型	不一定，一般 LLM 也可嘗試	通常需要模型本身具備相應機制
主要改變	輸入中的指令與示範	模型的訓練方式與推論行為
推論時間	因輸出更多步驟而增加	可主動投入更多計算量

2024 年的 o1 與 2025 年的 DeepSeek-R1，都是用大規模強化學習培養推理能力的路線；它們依然逐步生成 Token。



後訓練是階段， 微調是手段

POST-TRAINING

後訓練

預訓練之後的整段對齊流程，讓模型聽得懂指令、守規範、會推理。

FINE-TUNING

微調

更新權重的具體技術，是後訓練的主要手段，也能單獨做下游客製化。

GPT-1 的第二階段只是微調；SFT、RLHF 這類後訓練方法要到 2022 年才成形。



後訓練的三種方法：示範、成績、偏好

方法	TRAINING SIGNAL	WHAT IT LEARNS
SFT 監督微調	標準示範答案（問題＋正解）	老師示範解題：學會該怎麼回答
RL 強化學習	分數、對錯的獎勵訊號	做題目看成績：哪種策略容易成功
RLHF 人類回饋	人類對回答的偏好排序	老師說「A 比 B 好」：人類喜歡什麼
疊加順序	先 SFT 打底，再 RL／RLHF	示範建立格式，回饋校正偏好

ChatGPT 的關鍵是 SFT 加 RLHF；o1、DeepSeek-R1 則是把 RL 放大來練推理。



GPT-4 看世界，01 學思考

兩個方向同時演化：模型的輸入更廣（多模態），輸出前的思考更深（CoT）。

CONTEXT WINDOW = INPUT + REASONING（內心戲） + OUTPUT

2023 · OPENAI

MULTIMODAL

GPT-4

文字 · 8K → 128K context

圖像 · 看圖說故事、解 OCR、讀圖表

語音 / 視訊（GPT-4o）即時對話

BENCHMARK

律師資格考前 10% · GRE 數學滿分

GPT-3.5 同題落在後 10%

2024 · OPENAI

01 · 推理

PROMPT： 如果一個火車...（複雜推理題）

★ HIDDEN THINKING（內心戲） "Let me think... v1 = 60, v2 = 80...
wait, I should double-check..."

最終答案：兩列車在 1.5 小時後相遇

Test-time compute：給模型更多思考時間，AIME 從 13% → 83%。

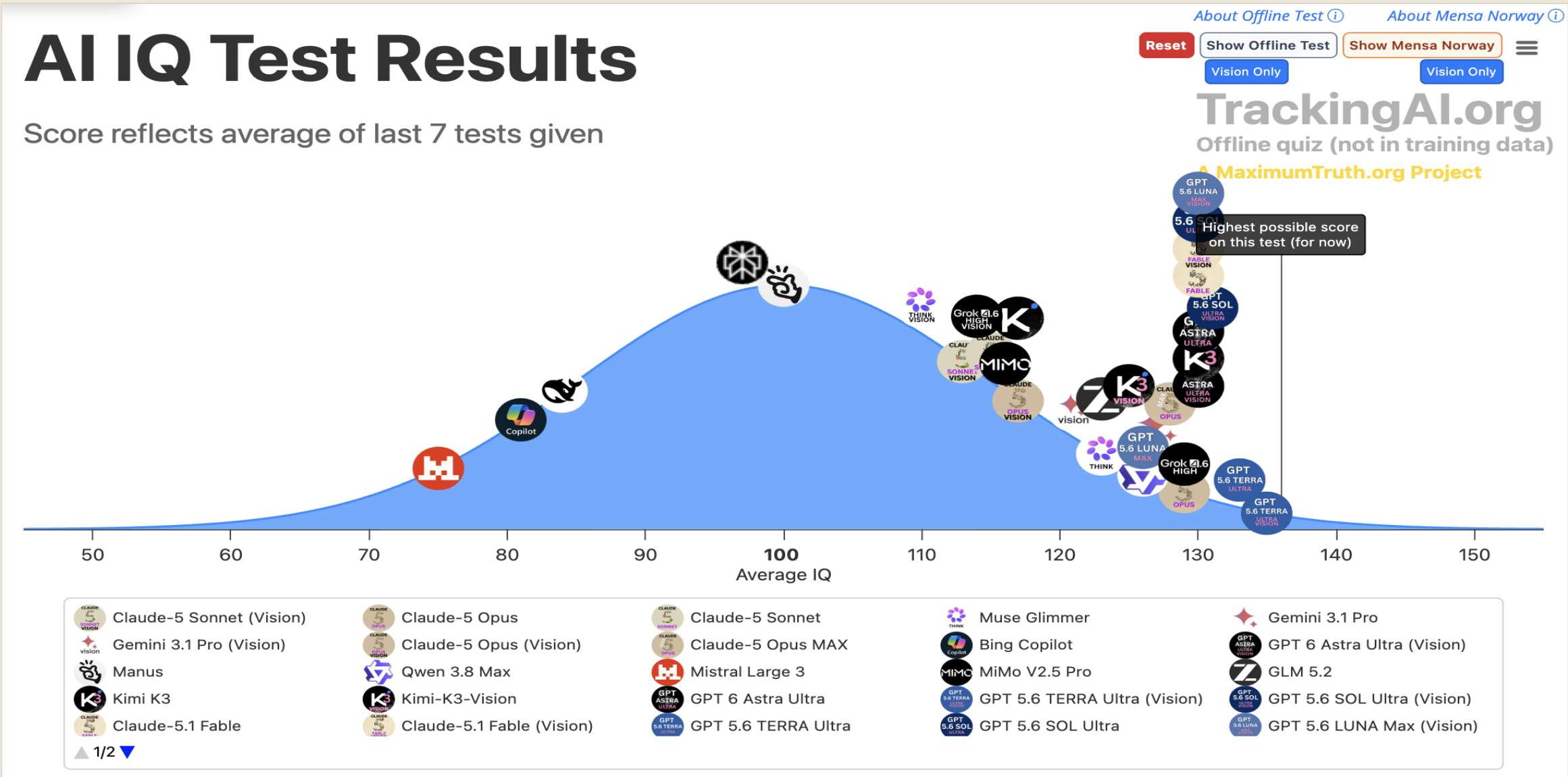
CH. 03 · GPT-4 / 01

buy me a coffee
需開街口支付掃描

33

AI IQ Test：模型的智商落在哪裡

分數為最近 7 次測驗的平均；題庫離線，不在模型的訓練資料裡。



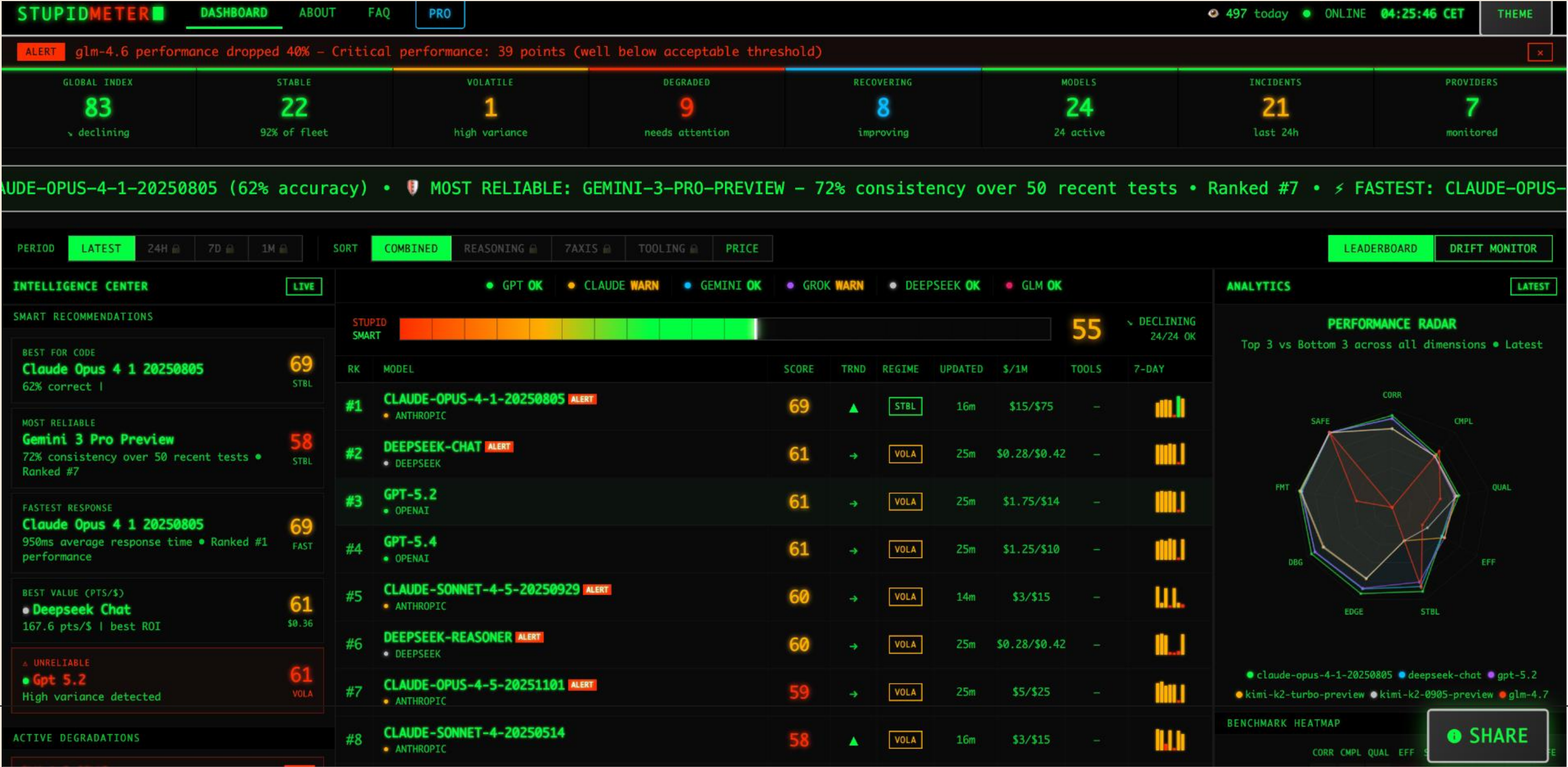
SOURCE · TRACKINGAI.ORG

<https://www.trackingai.org/home>



Stupid Meter：把「模型變笨」變成可以看的數字

社群做的即時看板：分數、波動、退化事件，全部攤在同一頁上。



打開地端模型，看它的內心戲

請問台南手搖飲的「半糖」，換算成台北的甜度標準大概是多少？

模型先把問題拆成「甜度基準」、「南北口味差異」與「換算假設」，還在草稿裡自己提醒：這只是飲食文化的認知，不是數學定律。

結論是台南半糖大致對應台北七到八分糖。步驟很完整，但換算仍是模型自己提出的假設。

觀察清單

01 它把問題拆成了哪些步驟？

02 中間有沒有猶豫、重複或改變解法？

03 推理過程與最後答案是否一致？

04 推理寫得很完整，答案就一定正確嗎？

05 換成一般模型直接回答，差別在哪？

「內心戲」是比喻。畫面上看到的是模型生成的 **推理軌跡**，不等於讀到神經網路內部全部真實的計算。



推理越長，就一定越正確嗎？

一開始的假設錯了，後面每一步都跟著走偏

產生看似合理、實際上無關的步驟

為錯誤答案補上一段非常有說服力的說明

在簡單問題上過度思考

推理過程很流暢，最後答案仍然錯誤

像考試時寫滿整張計算紙——寫得很多只能證明筆很努力，不一定代表答案正確。

需要外部資料的問題仍要查核來源；數學或程式問題可以用計算器、Python 或測試來驗證。



可見的推理，不等於真正透明

Chain-of-Thought 本身也是模型生成的文字。它可能是解題過程的一部分，也可能只是模型替答案整理出來的說明。所以看到推理之後，我們還要問：

每個步驟是否符合已知事實？

前後有沒有矛盾？

計算能否重新驗證？

引用的資料是否真的存在？

改一個條件，推論會合理地跟著變嗎？

它讓我們多了一個可以觀察的對象，但沒有讓查核責任消失。

補充

https://geominingical.github.io/knowledge-library/2025_karpathy/index.html



今天走過的三段路

01 任務從訓練流程，移進了我們提供的 Context。

02 資料整理的每個決定，都在決定模型會學到什麼。

03 要求 AI 拆解問題，其實也在練習自己的思考。



REFERENCES

Radford, A, et al. (2018). *Improving Language Understanding by Generative Pre-Training* .

Radford, A, et al. (2019). *Language Models are Unsupervised Multitask Learners* .

Brown, T. B., et al. (2020). *Language Models are Few-Shot Learners* . arXiv:2005.14165

Kaplan, J., et al. (2020). *Scaling Laws for Neural Language Models* . arXiv:2001.08361

Hoffmann, J., et al. (2022). *Training Compute-Optimal Large Language Models* . arXiv:2203.15556

Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in LLMs* . arXiv:2201.11903

Kojima, T., et al. (2022). *Large Language Models are Zero-Shot Reasoners* . arXiv:2205.11916

DeepSeek-AI. (2025). *DeepSeek-R1*. arXiv:2501.12948

Lanham, T., et al. (2023). *Measuring Faithfulness in Chain-of-Thought Reasoning* . arXiv:2307.13702

